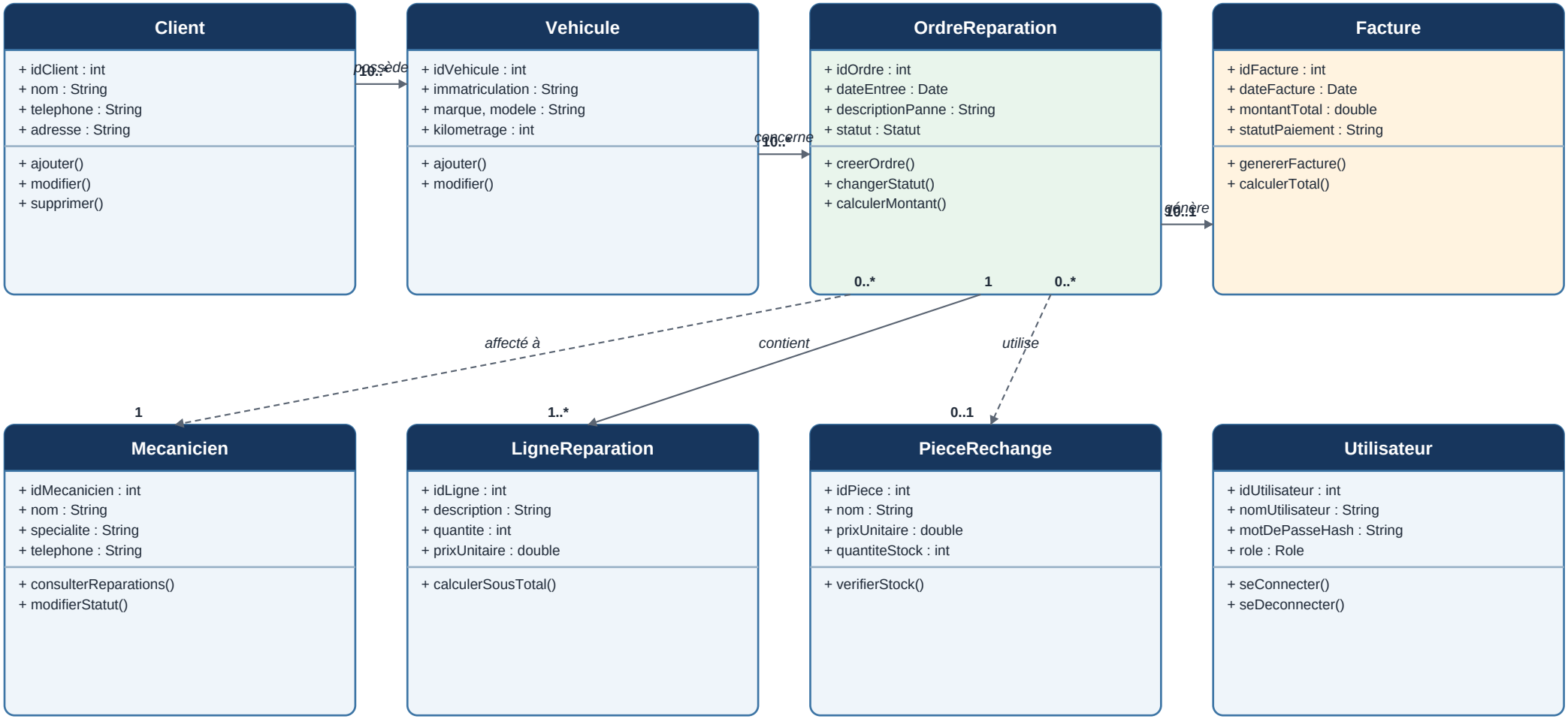
Diagramme de classes UML — GarageManager

Chaque rectangle contient : nom de classe / attributs / opérations



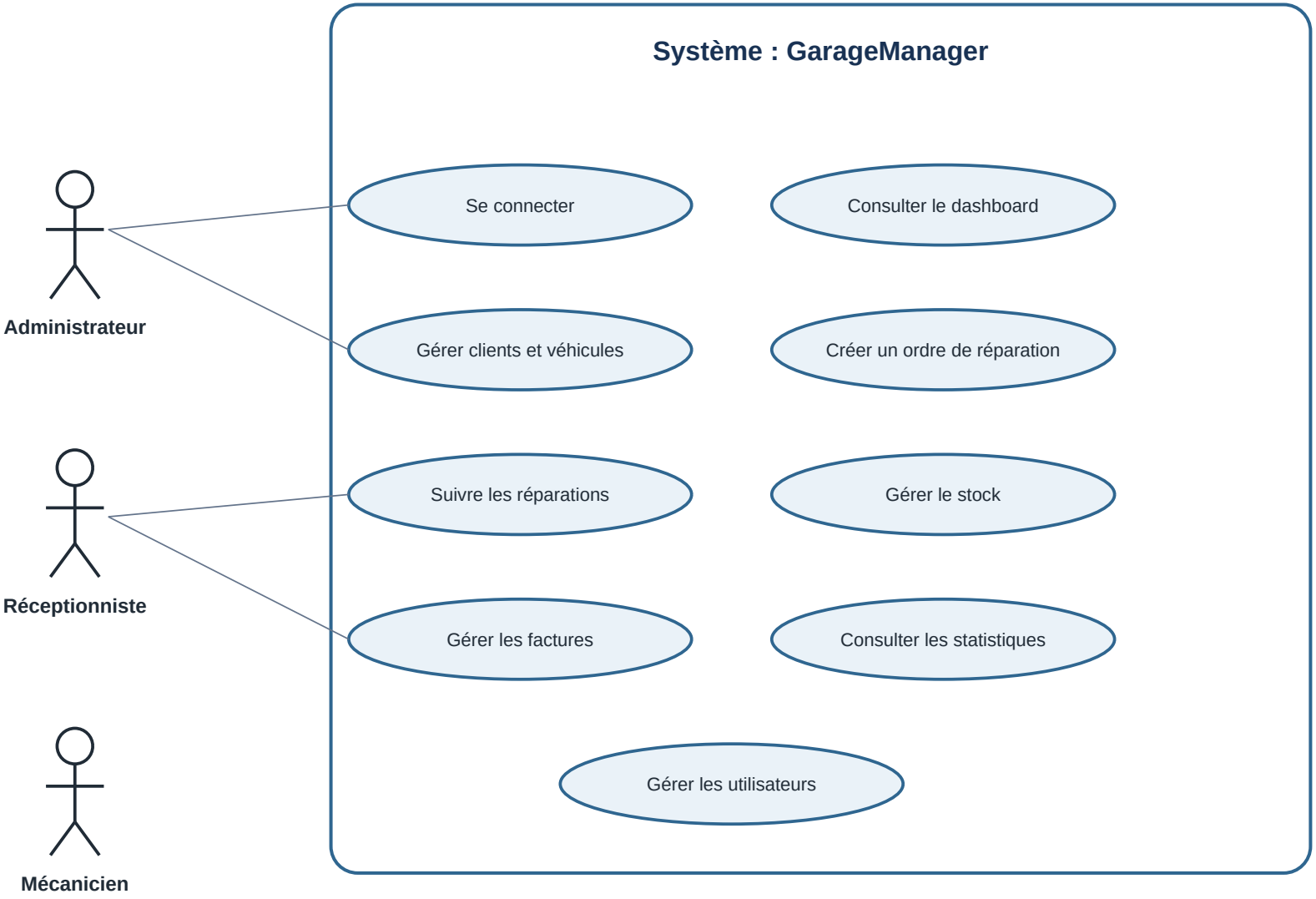
Statut : EN_ATTENTE • EN_COURS • TERMINEE • LIVREE | Role : ADMINISTRATEUR • RECEPTIONNISTE • MECANICIEN

Multiplicités indicatives : elles décrivent combien d'objets peuvent être liés; à valider selon les règles choisies par le groupe.

Mot de passe : stocker uniquement un hash sécurisé, jamais le texte en clair.

2. Diagramme de cas d’utilisation

Qui utilise l’application et quelles fonctions sont prévues ?



Associations à confirmer selon les rôles finaux.

Administrateur : utilisateurs, statistiques et configuration.

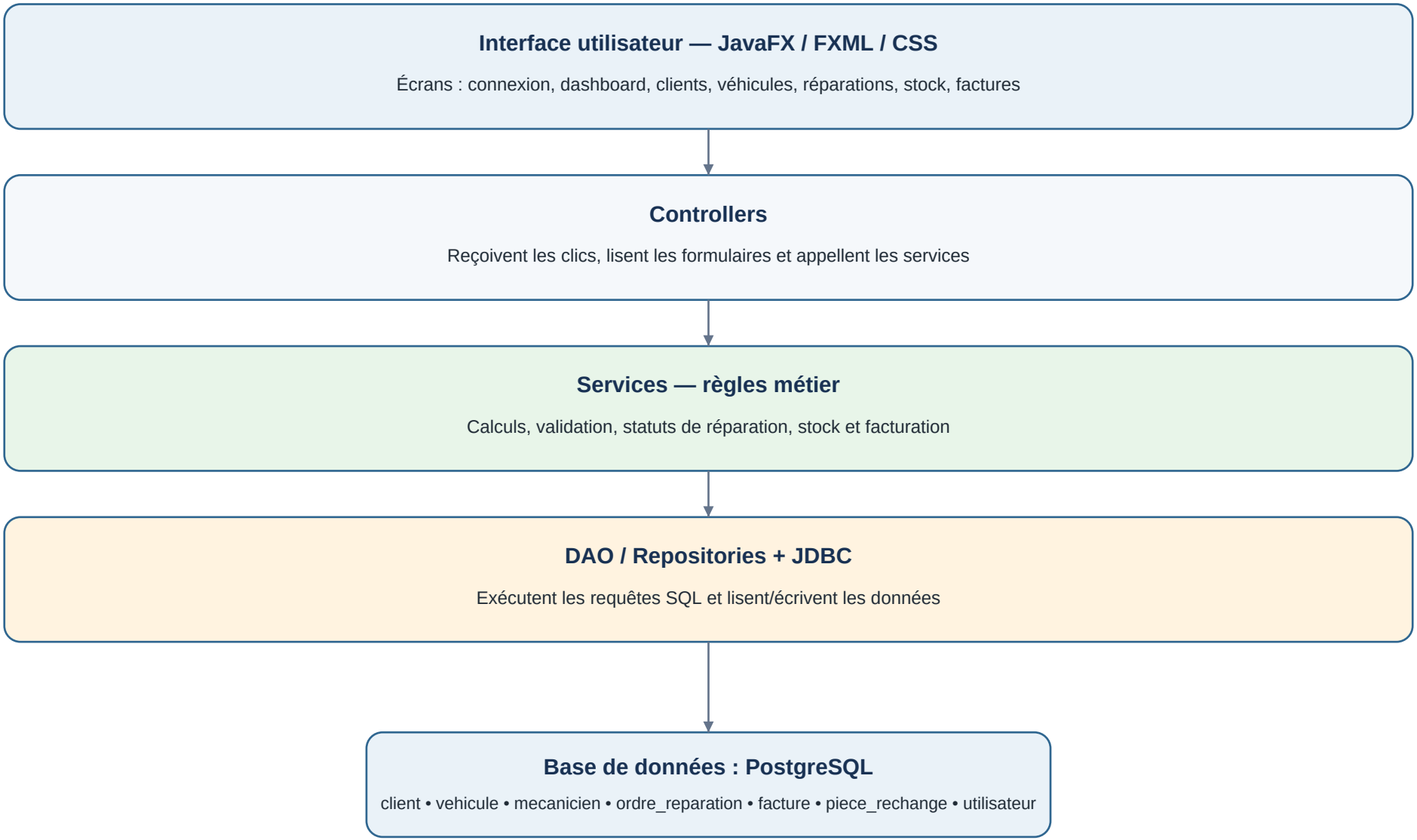
Réceptionniste : clients, véhicules, ordres et factures.

Mécanicien : consulter les ordres et changer leur statut.

Les liens indiquent les fonctions accessibles; prévoir les droits dans le code.

3. Architecture de l’application

Flux logique conseillé pour une application JavaFX connectée à PostgreSQL



JDBC permet aux classes d’accès aux données de communiquer avec PostgreSQL.

5. Vérifications à faire dans le diagramme de classes

Le diagramme de classes existant contient Client, Véhicule, OrdreReparation, Facture, Utilisateur, Statut, Mecanicien, LigneReparation et PieceRechange. Avant de le considérer final, vérifier les points suivants :

• Ajouter les identifiants de relation clairement : Client 1 — 0..* Véhicule; Véhicule 1 — 0..* OrdreReparation.

• Clarifier la relation OrdreReparation–Facture : 1 ordre vers 0..1 facture si la règle choisie est une facture maximum par ordre.

• Ajouter une association explicite entre OrdreReparation et Mecanicien (par exemple un mécanicien affecté à plusieurs ordres; le nombre exact dépend de votre règle métier).

• Une LigneReparation devrait être rattachée à un OrdreReparation. Si une ligne représente une pièce, relier la ligne à PieceRechange; si elle peut représenter la main-d'œuvre aussi, préciser le type de ligne ou séparer les concepts.

• Le rôle d'Utilisateur devrait idéalement être un type contrôlé (énumération Role) plutôt qu'une chaîne libre; le mot de passe doit être stocké sous forme de hash, pas en clair.

• Ajouter si nécessaire à Facture : numeroFacture, dateEcheance (facultative), montantPaye ou statut de paiement clairement défini.

• Ajouter à OrdreReparation les dates réelles de début/fin si elles sont utiles au suivi.

• Vérifier que chaque association possède les multiplicités aux deux extrémités et que les noms des classes sont cohérents avec les tables SQL.

Exemple de modèle relationnel à envisager

Table	Champs principaux (proposition)
client	id_client PK, nom, telephone, adresse
vehicule	id_vehicule PK, immatriculation UNIQUE, marque, modele, kilometrage, id_client FK
utilisateur	id_utilisateur PK, nom_utilisateur UNIQUE, mot_de_passe_hash, role, actif
mecanicien	id_mecanicien PK, nom, specialite, telephone
ordre_reparation	id_ordre PK, id_vehicule FK, id_mecanicien FK (si affectation unique), date_entree, date_sortie_prevue, date_debut, date_fin, description_panne, statut
piece_rechange	id_piece PK, nom, prix_unitaire, quantite_stock, seuil_alerte
ligne_reparation	id_ligne PK, id_ordre FK, id_piece FK nullable si main-d'œuvre possible, description, quantite, prix_unitaire
facture	id_facture PK, id_ordre FK UNIQUE si une seule facture par ordre, date_facture, montant_total, statut_paiement

PK = clé primaire; FK = clé étrangère. C'est une proposition de structure, à ajuster au diagramme UML final et aux décisions du groupe.